

AI-POWERED

Intelligent Semantic Dictionary & Search Engine

From Classical DSA Foundations to AI-Powered Semantic Retrieval

Trie Engine

FAISS Vector Search

RAG Pipeline

FastAPI Backend

LLM Integration

Document Type:

Technical Project Specification & Architecture Blueprint

Target Audience:

SDE Internship Candidates | Senior Technical Reviewers

Build Estimate:

12–16 Weeks | 3,000–6,000 Lines of Code

Stage Coverage:

DSA Core → REST Backend → AI Semantic Retrieval

■

TABLE OF CONTENTS

#	Section	Coverage
01	Project Vision	Concept, transformation arc, value proposition
02	Final Features	15 core features with flows and complexity
03	System Architecture	High-level design, request lifecycle, components
04	DSA Foundations	Trie, Graphs, Edit Distance, Priority Queues
05	Backend Engineering	FastAPI, async design, API structure, error handling
06	Database & Storage Design	SQLite/PostgreSQL schema, Redis, FAISS, ChromaDB
07	AI / LLM Layer	Embeddings, FAISS, RAG pipeline, LLM abstraction
08	Implementation Roadmap	7-phase, 14-week build plan with tasks
09	Complete Tech Stack	All technologies with rationale and relevance
10	Engineering Challenges	8 real performance & design challenges solved
11	Internship & Resume Value	Talking points, bullets, GitHub strategy
12	Future Extensions	8 advanced extensions for scale and features

01 PROJECT VISION

1.1 The Core Idea

The **AI-Powered Intelligent Semantic Dictionary** is not a conventional dictionary app. It is a complete **language intelligence platform** that begins with classical computer science fundamentals — Trie-based autocomplete and graph-based synonym traversal — and evolves into a modern AI retrieval system capable of understanding query intent, computing semantic similarity, and generating context-aware explanations using LLM APIs.

1.2 The Transformation Arc

Stage	Name	Core Technologies	Deliverable
Stage 1	Classical Dictionary (DSA Focus)	Trie, HashMap, BFS/DFS, Levenshtein Distance	CLI dictionary with autocomplete & fuzzy search
Stage 2	Intelligent Search Engine (Backend Engineering)	FastAPI, SQLite/PostgreSQL, Redis, Pydantic	REST API with caching, async request handling
Stage 3	Semantic AI Retrieval (AI/LLM Engineering)	SentenceTransformers, FAISS, RAG, LLM APIs	Meaning-aware search with LLM-generated explanations

Each stage is independently deployable and demonstrable — you always have something working and presentable regardless of build progress.

1.3 Why This Project Is Technically Valuable

Discipline	What You Build	Industry Analog
Data Structures & Algorithms	Trie, Graph, BFS/DFS, Edit Distance	Compiler internals, DNS resolution
Backend Systems Engineering	FastAPI, REST, Caching, DB Design	Microservice backends at scale
AI/ML Systems	Embeddings, Vector Search, RAG, LLMs	Semantic search at Elasticsearch, Google, OpenAI

Why This Beats a Typical CRUD Project: Most student projects are todo apps or shallow CRUD APIs. This project forces you to implement a Trie from scratch, model a synonym graph, compute real semantic embeddings, design a RAG retrieval pipeline, and reason about latency and caching — the exact conversations a senior engineer has in a system design interview.

1.4 Internship-Level Positioning

■	Demonstrate understanding of data structures beyond textbook level
■	Show ability to build and expose a real API
■	Prove AI/ML-awareness without overclaiming expertise

02 FINAL FEATURES

#	Feature	Key Algorithm	Complexity	Status
01	Word Search	Trie + HashMap	O(L)	Core MVP
02	Autocomplete	Trie BFS + Frequency Rank	O(L + W log W)	Core MVP
03	Fuzzy Search / Typo Correction	Levenshtein DP + BK-Tree	O(log N) avg	Core MVP
04	Synonym Graph Traversal	BFS/DFS on Adjacency List	O(V+E)	Phase 4
05	Word Relationship Graph API	Graph → JSON for D3.js	O(V+E)	Phase 4
06	AI Simple Explanations	LLM prompt engineering	API call	Phase 6
07	Contextual Sentence Gen.	LLM with caching (Redis 7d)	API call	Phase 6
08	Multilingual Semantic Support	LaBSE (109 languages)	O(D)	Phase 6
09	Embedding-Based Similarity	SentenceTransformer + cosine	O(N×D)	Phase 5
10	Vector Search	FAISS IndexFlatIP / HNSW	O(log N)–O(N)	Phase 5
11	RAG-Style Retrieval	Embed → FAISS → LLM prompt	O(log N) + LLM	Phase 6
12	Semantic Query Understanding	Regex + LLM intent parser	O(1)	Phase 6
13	Multi-Signal Ranking	Weighted scoring formula	O(k log k)	Phase 6
14	React Dashboard Frontend	React + D3.js + Debounce	Frontend	Phase 3
15	Scalability Extensions	Elasticsearch, sharding, voice	Future	Phase 7+

Feature Deep-Dives

Feature 2: Autocomplete Using Trie

User types 'epi' → system suggests ['epic', 'episode', 'epidemic', 'epiphany', 'epitome'] via BFS traversal from prefix endpoint node, ranked by search frequency. Debounced at 300ms on frontend.

Feature 3: Fuzzy Search / Typo Correction

User types 'epifany' → Levenshtein edit distance DP computes minimum operations to reach 'epiphany' (distance=2). BK-Tree prunes 70-90% of candidates without checking them.

Feature 9: Embedding-Based Semantic Similarity

Query: 'melancholy' → semantic neighbors: sadness(0.94), sorrow(0.93), wistfulness(0.89), longing(0.87), nostalgia(0.85). Richer than synonym graphs — captures thematic clusters from LLM training.

Feature 11: RAG-Style Retrieval

Query: 'word for feeling happy at someone else's misfortune' → embeds query → FAISS retrieves [schadenfreude, malice, envy] → definitions assembled as context → LLM generates grounded answer.

Feature 13: Multi-Signal Ranking Formula

```
Final Score =  $\alpha \times \text{semantic\_similarity}$   
            +  $\beta \times \text{word\_frequency\_score}$   
            +  $\gamma \times \text{exact\_prefix\_match\_bonus}$   
            +  $\delta \times \text{recency\_score}$  (recently searched)  
            -  $\epsilon \times \text{word\_complexity\_penalty}$ 
```

Default weights: $\alpha=0.5$, $\beta=0.2$, $\gamma=0.2$, $\delta=0.1$, $\epsilon=0.0$ (configurable)

03 SYSTEM ARCHITECTURE

3.1 High-Level Architecture Overview

Layer	Components & Responsibilities
CLIENT LAYER	React Frontend / curl / Postman / Mobile App Search input with debounced autocomplete • Result display with relationship graph • AI explanation panel
API GATEWAY (FastAPI + Uvicorn)	Request routing & validation (Pydantic) • Rate limiting middleware CORS configuration • Error handling • Authentication
SEARCH ENGINE	• Trie Engine (prefix search, autocomplete) • Fuzzy Engine (Levenshtein, BK-Tree) • Ranking Service
GRAPH ENGINE	• Synonym/Antonym/Hypernym graph • BFS/DFS traversal • WordNet data source
AI / EMBEDDING LAYER	• SentenceTransformer model • FAISS vector index • LLM client (OpenAI / Ollama) • ChromaDB (optional)
STORAGE LAYER	SQLite / PostgreSQL (word records) • Redis (hot-path cache) FAISS index file • ChromaDB persistent store
EXTERNAL SERVICES	OpenAI / Gemini API • WordNet (NLTK, local) • HuggingFace Inference API

3.2 Component Responsibilities

Component	Primary Responsibility	Technology
Trie Engine	Prefix search, autocomplete, existence check	Pure Python TrieNode class
Fuzzy Engine	Typo correction via edit distance	Python + BK-Tree
Graph Engine	Synonym/antonym/hypernym traversal	NetworkX or adjacency dict
Embedding Engine	Vector encoding of words/queries	SentenceTransformers
Vector Store	ANN search over embedding space	FAISS (local) / ChromaDB
LLM Client	Text generation, explanation, RAG	OpenAI API / Ollama
Word Repository	Persistent word/definition storage	SQLAlchemy + SQLite/PostgreSQL
Cache Layer	Hot-path response caching	Redis
API Layer	Request routing, validation, response	FastAPI + Pydantic
Query Parser	Intent classification, entity extraction	Regex + optional LLM

3.3 Request Lifecycle Trace — 'meloncolly' (misspelled query)

Step	Action	Detail
Step 1	Client sends HTTP GET	GET /api/v1/search?q=meloncolly
Step 2	FastAPI routing & validation	SearchController.handle_search() Pydantic validates query
Step 3	Intent classification	QueryParser.classify() → Intent(type='word_lookup')
Step 4	Trie exact match fails	TrieEngine.search('meloncolly') → False → trigger fuzzy
Step 5	Fuzzy correction	FuzzyEngine.correct() → 'melancholy' (edit_distance=2, conf=0.89)
Step 6	DB lookup (cache miss)	Redis.get('word:melancholy') → miss → WordRepository.get()
Step 7	Parallel async calls	asyncio.gather(graph_related, semantic_neighbors)
Step 8	Response assembly	Structured JSON with correction, definition, related, neighbors
Step 9	Cache write	Redis.setex('word:melancholy', ttl=3600, value=response)
Step 10	HTTP 200 response	Complete response returned to client

04 DSA FOUNDATIONS

4.1 Trie (Prefix Tree)

A Trie (from **re-trie-val**, pronounced 'try') is a tree-based data structure optimized for string operations. All strings sharing a common prefix share a common path from the root — enabling O(L) lookup, O(L) prefix search, and natural alphabetical ordering.

TrieNode Implementation:

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, 'TrieNode'] = {} # char → TrieNode
        self.is_end_of_word: bool = False
        self.frequency: int = 0 # search frequency for ranking
        self.word_id: Optional[int] = None # FK to database record

# dict-based children preferred over array[26] for:
# 1. Unicode support 2. Sparse vocab 3. Pythonic style
```

Memory Comparison for 100,000 English Words:

Implementation	Est. RAM	Notes
dict-based children	~80–120 MB	Flexible, Unicode, sparse-friendly — RECOMMENDED
array[26] children	~200–400 MB	Fixed ASCII only, wastes space on leaf nodes
Compressed (Radix)	~20–40 MB	Complex to implement, best memory efficiency
Patricia Trie	~15–30 MB	Most memory-efficient, hardest to build correctly

4.1.1 Trie Autocomplete BFS Algorithm:

```
def collect_words(self, node, prefix, limit):
    results, queue = [], deque()
    queue.append((node, prefix))
    while queue and len(results) < limit:
        current_node, current_word = queue.popleft()
        if current_node.is_end_of_word:
            results.append((current_word, current_node.frequency))
        sorted_children = sorted(
            current_node.children.items(),
            key=lambda x: x[1].frequency, reverse=True) # best-first by freq
        for char, child in sorted_children:
            queue.append((child, current_word + char))
    return sorted(results, key=lambda x: x[1], reverse=True)
```

4.1.2 Trie Complexity Summary:

Operation	Time Complexity	Space Complexity
Insert	$O(L)$	$O(L)$ worst case
Search (exact)	$O(L)$	$O(1)$
Prefix check	$O(L)$	$O(1)$
Autocomplete (BFS)	$O(L + W \log W)$	$O(W)$
Build (N words)	$O(N \times \text{avg_L})$	$O(N \times \text{avg_L})$

4.2 Levenshtein Edit Distance (DP)

The **edit distance** between two strings S and T is the minimum number of single-character operations (insert, delete, substitute) to transform S into T. Computed via dynamic programming in $O(|S| \times |T|)$ time.

Example DP Table — 'epifany' vs 'epiphany' (edit distance = 2):

		e	p	i	p	h	a	n	y
	0	1	2	3	4	5	6	7	8
e	1	0	1	2	3	4	5	6	7
p	2	1	0	1	2	3	4	5	6
i	3	2	1	0	1	2	3	4	5
f	4	3	2	1	1	2	3	4	5
a	5	4	3	2	2	2	2	3	4
n	6	5	4	3	3	3	3	2	3
y	7	6	5	4	4	4	4	3	

Edit distance = 2 (substitute 'f'→'p', insert 'h'). Below threshold=2 → 'epiphany' suggested.

4.3 Synonym Graph (BFS Traversal)

Word relationships (synonym, antonym, hypernym, hyponym) are modeled as a weighted directed graph using an adjacency list (dict of dicts). BFS with depth limit finds the most related words first, while Dijkstra's algorithm finds the highest-weight semantic path between two words.

Graph Traversal — 'joy' at depth 2:

Word	Depth	Relation	Weight
happiness	1	synonym	0.95
elation	1	synonym	0.88
grief	1	antonym	0.90
contentment	2	synonym	0.82
euphoria	2	synonym	0.80

4.4 Comprehensive Complexity Table

Feature	Data Structure	Time	Space
Word lookup	Trie + HashMap	$O(L)$	$O(1)$
Autocomplete	Trie BFS	$O(L + W \log W)$	$O(W)$
Fuzzy correction	Edit distance + BK-Tree	$O(\log N)$ avg	$O(N)$
Synonym lookup	Graph adjacency	$O(1)$	$O(1)$
Related words BFS	Graph BFS	$O(V+E)$	$O(V)$
Shortest path	Dijkstra	$O((V+E) \log V)$	$O(V)$
Embedding search	FAISS ANN	$O(\log N)$	$O(N \times D)$
Exact vector search	FAISS Flat	$O(N \times D)$	$O(N \times D)$
DB word fetch	SQL index	$O(\log N)$	$O(1)$
Cache lookup	Redis GET	$O(1)$	$O(1)$

05 BACKEND ENGINEERING

5.1 Framework Selection: FastAPI

Criterion	FastAPI ■	Flask	Django
Performance	Async, ASGI, very fast	Sync (WSGI, slower)	Sync (WSGI)
Type safety	Pydantic built-in	Manual	Partial
Auto docs	OpenAPI/Swagger auto	Manual/extension	Manual
Async support	Native (asyncio)	Requires extensions	Django 3.1+
Production use	Uber, Netflix, Microsoft	Millions of apps	Instagram, Pinterest

5.2 REST API Endpoint Map

Category	Method	Endpoint	Description
Search	GET	/api/v1/search?q={query}	Main search: exact + semantic
Search	GET	/api/v1/word/{word}	Full word record
Search	GET	/api/v1/autocomplete?prefix={p}	Trie autocomplete suggestions
Search	GET	/api/v1/fuzzy?q={query}	Fuzzy correction candidates
Semantic	GET	/api/v1/semantic/similar?word={w}&k={n}	Top-k semantic neighbors
Semantic	POST	/api/v1/semantic/query	Natural language semantic query
Graph	GET	/api/v1/graph/related?word={w}&depth={d}	BFS synonym traversal
Graph	GET	/api/v1/graph/path?from={w1}&to={w2}	Shortest semantic path
AI	GET	/api/v1/ai/explain?word={w}&level={l}	LLM-generated explanation
AI	POST	/api/v1/ai/rag	RAG-style NL question answering
Admin	POST	/api/v1/admin/rebuild-index	Rebuild FAISS index
Health	GET	/health	Service health check

5.3 Async Parallel Request Handling

```
@router.get('/search')
async def search(q: str, include_semantic: bool = True, ...):
    cached = await redis.get(f'search:{q}')
    if cached: return json.loads(cached)
    # Run independent operations CONCURRENTLY
    tasks = [
        asyncio.create_task(get_word_definition(q, db)),
        asyncio.create_task(get_graph_related(q)),
    ]
    if include_semantic:
        tasks.append(asyncio.create_task(get_semantic_neighbors(q)))
    results = await asyncio.gather(*tasks, return_exceptions=True)
    response = assemble_response(q, results)
    await redis.setex(f'search:{q}', 3600, json.dumps(response))
    return response
```

06 DATABASE & STORAGE DESIGN

6.1 Storage Architecture Overview

Data Type	Volume Estimate	Access Pattern	Storage Solution
Word records (text, definition, POS)	~170K records	Read-heavy, random	SQLite → PostgreSQL
Word embeddings (768-dim float32)	~500MB	Batch read at startup	FAISS index file
Synonym graph edges	~2M edges	Read at startup, in-mem	JSON → SQLite
LLM-generated content cache	Variable	Read >> write	Redis + DB persist
Search frequency counters	170K counters	Frequent writes	Redis
User query logs (optional)	Growing	Append-only	SQLite / PostgreSQL

6.2 Database Schema (Key Tables)

```
CREATE TABLE words (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  word        TEXT NOT NULL UNIQUE,  
  phonetic    TEXT,           -- IPA phonetic notation  
  pos         TEXT NOT NULL,  -- part of speech  
  etymology   TEXT,  
  frequency   INTEGER DEFAULT 0,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_words_word ON words(word);  
  
CREATE TABLE word_relations (  
  word_id      INTEGER REFERENCES words(id),  
  related_word_id INTEGER REFERENCES words(id),  
  relation_type TEXT NOT NULL,  -- synonym/antonym/hypernym/hyponym  
  weight       REAL DEFAULT 1.0  
);
```

6.3 Redis Cache Key Namespacing

Cache Key Pattern	Value	TTL
search:{query}	JSON SearchResult	1 hour
autocomplete:{prefix}:{limit}	JSON suggestions list	30 min
word:{word}	JSON WordDefinition	6 hours
ai:explain:{word}:{level}	AI explanation string	7 days
ai:examples:{word}	AI example sentences JSON	7 days
semantic:similar:{word}:{k}	JSON similar words list	1 hour

freq:{word}	Integer search counter	No TTL
-------------	------------------------	--------

6.4 Vector Storage Comparison

Feature	FAISS	ChromaDB	Pinecone
Type	Library (in-process)	Embedded DB	Cloud service
Persistence	Manual (.index files)	Built-in (SQLite)	Cloud-managed
Metadata filtering	No	Yes	Yes
Scale tested	Billions (Meta)	Millions	Billions
Cost	Free	Free	Paid after free tier
Best for	Research + internship	Dev/small prod	Production SaaS

07 AI / LLM LAYER

7.1 What Are Embeddings?

An embedding maps discrete symbolic input (a word, sentence, document) to a dense vector of real numbers in high-dimensional space, trained such that **semantically similar inputs map to geometrically nearby vectors**. 'cat' and 'kitten' end up close; 'cat' and 'airplane' are far apart.

Word2Vec vs SentenceTransformers:

Property	Word2Vec (2013)	SentenceTransformers (SBERT) ■
Granularity	One vector per word type	One vector per sentence/phrase
Context-aware?	No — 'bank' same regardless	Yes — financial vs river bank differ
Dimensions	300	384 or 768
Multi-word queries	Poor	Excellent
Best for	Word-level similarity	Semantic similarity tasks ← USE THIS

7.2 SentenceTransformer Pipeline:

```
Input: "The bank approved my loan application"
↓
Tokenization (WordPiece/BPE):
["The", "bank", "approved", "my", "loan", "application"]
↓
BERT Encoder (12 transformer layers):
Each token → contextual embedding [768-dim]
Token 'bank' gets FINANCIAL-context vector here
↓
Mean Pooling (average all token embeddings):
sentence_vector = mean([t1, t2, ..., tn]) → float32[768]
↓
L2 Normalization:
sentence_vector = sentence_vector / ||sentence_vector||
↓
Output: Dense vector float32[768] ← meaning of the sentence
```

7.3 Cosine Similarity — The Metric

Cosine similarity measures the **angle** between two vectors, not the distance. This normalizes out sentence length differences — a 10-word sentence and a 3-word sentence with the same meaning will still have high similarity. Range: -1 (opposite) to +1 (identical).

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Where: $A \cdot B = \sum(A_i \times B_i)$ ← dot product

$||A|| = \sqrt{\sum(A_i^2)}$ ← L2 norm

Result range: +1.0 = semantically identical

0.0 = semantically unrelated

-1.0 = semantic opposites (approximate)

FAISS trick: normalize vectors first → inner product == cosine similarity

7.4 Model Selection Guide

Model	Dim	Size	Speed	Quality	Use Case
all-MiniLM-L6-v2 ■	384	90MB	Fast	Good	Default English — recommended
all-mpnet-base-v2	768	420MB	Medium	Better	Higher quality English
LaBSE	768	1.8GB	Slow	Best multilingual	Cross-lingual retrieval (109 langs)
paraphrase-multilingual-MiniLM	384	420MB	Fast	Good	Multilingual, balanced
text-embedding-3-small (OpenAI)	1536	Cloud	API latency	Excellent	API-based, highest quality

7.5 RAG Pipeline — Step by Step

RAG (Retrieval-Augmented Generation) retrieves relevant context, then augments an LLM prompt with that context to generate a grounded response — preventing hallucination by constraining answers to retrieved facts.

Step	Action
Query	User: 'word for feeling happy at someone else's misfortune'
Embed	query_vector = encode(query) → float32[384]
Retrieve	faiss.search(query_vector, k=5) → [schadenfreude(0.91), malice(0.89), envy(0.87), gloating(0.86), spite(0.83)]
Hydrate	Fetch full definitions for top-3 from DB → assemble context string
Prompt	Build prompt: 'Based on these definitions, answer: {query}\nEntries: {context}\nAnswer:'
Generate	LLM.complete(prompt, temp=0.3) → 'The word is schadenfreude — borrowed from German...'
Cache + Return	Redis.setex(key, 3600, response) → HTTP 200 to client

08 IMPLEMENTATION ROADMAP

Phase 1 Weeks 1–2

Base DSA Dictionary

- Implement TrieNode + Trie: insert, search, autocomplete (BFS), delete
- Levenshtein DP: compute(s, t) with full table, brute-force correction
- Unit tests for all Trie operations including edge cases (unicode, empty)
- CLI interface: `python cli.py search <word>` / `python cli.py autocomplete <prefix>`
- Load 500–1000 word test vocabulary from file

Phase 2 Weeks 3–4

Backend APIs

- FastAPI project structure (see Section 5.7 folder layout)
- SQLite schema from Section 6.2 + `scripts/load_dictionary.py`
- Endpoints: `/word/{word}`, `/autocomplete`, `/fuzzy`, `/search`
- Pydantic schemas, dependency injection, startup Trie build
- Swagger UI at `/docs` — instant demo-ability

Phase 3 Weeks 5–6

Frontend (React)

- Vite React app with 300ms debounce search hook
- Autocomplete dropdown (`position: absolute` below search bar)
- WordCard component: definition, POS, phonetic, examples
- 'Did you mean?' banner for fuzzy-corrected queries

Phase 4 Weeks 7–8

Graph Intelligence

- NLTK WordNet setup: `nlk.download('wordnet')`
- `scripts/populate_graph.py` → extract relations → `word_relations` table
- GraphEngine: `get_neighbors`, `bfs_related`, `dijkstra_path`
- Endpoints: `/graph/related`, `/graph/path`, `/graph/neighbors`

Phase 5 Weeks 9–10

AI Embeddings

- `scripts/generate_embeddings.py` — 10–30 min for 50K words on CPU
- `scripts/build_faiss_index.py` — IndexFlatIP for <50K vocab
- EmbeddingEngine class: `encode`, `encode_batch`, `find_similar`
- Endpoint: `/semantic/similar?word=...&k=...` — test manually

Phase 6 Weeks 11–12

Semantic Retrieval & RAG

- `POST /semantic/query` for natural language queries
- Full RAG pipeline: `embed` → `FAISS` → `hydrate context` → `LLM`
- LLM client (Ollama zero-cost first, then OpenAI option)
- Prompt templates for explain, examples, definitions
- Redis caching for all LLM responses (7-day TTL)

Phase 7 Weeks 13–14

Deployment & Optimization

- Dockerfile + docker-compose.yml (api + redis services)
- Deploy to Railway.app or Render.com (free tier Docker)
- GitHub Actions CI: run pytest on every push
- Profile slow endpoints, tune FAISS nprobe, add connection pooling

09 COMPLETE TECH STACK

Technology	Category	Purpose	Internship Value
Python 3.11+	Language	Backend implementation	Very High
FastAPI	Web Framework	REST API server (async-native)	High
SQLAlchemy 2.0	ORM	Database abstraction (type-safe)	High
SQLite → PostgreSQL	Database	Persistent word storage	High
Redis	Cache	Response & frequency caching	High
SentenceTransformers	ML Library	Embedding generation	Very High
FAISS	Vector Search	ANN similarity search (Meta)	High
ChromaDB	Vector DB	Persistent vector storage (alt)	High
NLTK + WordNet	NLP Library	Synonym/antonym graph data	Medium
OpenAI API	LLM	Text generation, explanations	Very High
Ollama	Local LLM	Offline LLM inference (free)	Medium
React + Vite	Frontend	User interface (component model)	High
D3.js	Visualization	Word relationship force graph	Medium
Docker	DevOps	Containerized deployment	Very High
pytest	Testing	Unit + integration tests	High
Pydantic v2	Validation	Request/response schemas	High

10 ENGINEERING CHALLENGES

Challenge 1 Embedding Generation Latency

Problem: SentenceTransformer encoding takes 50–200ms per query on CPU — too slow for <100ms real-time API target.

- ✓ Pre-compute all vocabulary embeddings offline (only query encoded at runtime ~20ms)
- ✓ In-process embedding cache (dict) — common queries served in <1ms
- ✓ Smaller model: all-MiniLM-L6-v2 (90MB, ~30ms) vs all-mpnet (420MB, ~150ms)
- ✓ GPU inference via CUDA — reduces encoding from 50ms to <5ms

Challenge 2 Trie Memory Explosion

Problem: Full English dictionary (170K words) with dict-based Trie nodes consumes ~150–300MB RAM.

- ✓ Vocabulary pruning: load only top 50,000 most frequent words into RAM Trie
- ✓ Radix/Patricia Trie: compress single-child paths — reduces memory by 40–60%
- ✓ Trie serialization: build once, serialize to disk, load at startup

Challenge 3 API Latency Composition

Problem: LLM calls take 800ms+ which is unacceptable for a synchronous search response.

- ✓ Return search results immediately (Trie + DB, <50ms) — LLM loads asynchronously
- ✓ Redis cache with 7-day TTL for LLM outputs — 90%+ cache hit on repeat queries
- ✓ Server-Sent Events (SSE) for streaming LLM responses progressively
- ✓ Trigger LLM only on explicit 'Explain' click, not on every search

Challenge 4 Semantic Ambiguity

Problem: 'Bank' → financial institution or river bank? Single embedding averages both meanings.

- ✓ POS disambiguation: user specifies 'bank (noun, financial)' vs 'bank (verb, to turn)'
- ✓ Context-aware search: full query string ('bank deposit') disambiguates better than word alone
- ✓ Multiple sense embeddings: WordNet synsets → one embedding per sense

Challenge 5 Typo Correction at Scale

Problem: Levenshtein brute force against 50K words = 3.2M operations per query (~50ms).

- ✓ BK-Tree: exploits metric property to prune 70–90% of words without checking
- ✓ Phonetic hashing (Soundex/Metaphone): group by pronunciation, check bucket first
- ✓ Trie-based pruning: traverse while tracking accumulated edit distance, prune early

Challenge 6 Caching Strategy Tradeoffs

Problem: Different layers provide different hit rates and latency tradeoffs.

- ✓ In-process dict: ~40% hit rate, 0ms latency, cleared on app restart
- ✓ Redis: ~85% hit rate after warmup, 1–5ms latency, TTL-based invalidation
- ✓ Layered cache: in-process → Redis → DB → write to both caches on miss

11

INTERNSHIP & RESUME VALUE

11.1 Resume Bullet Points

- Designed & built an AI-powered semantic dictionary with 3-tier architecture: Trie-based autocomplete ($O(L)$), WordNet synonym graph traversal, and FAISS vector search over SentenceTransformer embeddings for 50K+ words.
- Implemented Retrieval-Augmented Generation (RAG) pipeline: NL query → embedding-based retrieval → GPT-4o-mini generation — grounding responses in retrieved dictionary definitions to reduce hallucination.
- Engineered FastAPI REST backend with async request handling, Redis caching (85%+ hit rate), and Pydantic-validated schemas — handling concurrent Trie lookups, graph traversals, and LLM calls in <50ms.
- Built custom Trie with BFS autocomplete and Levenshtein-based fuzzy correction; analyzed memory-performance tradeoffs of dict-based vs compressed (Radix) Trie representations.
- Deployed containerized service (Docker + docker-compose) with GitHub Actions CI; integrated multilingual semantic search via LaBSE supporting cross-lingual retrieval across 50+ languages.

11.2 Interview Talking Points

Q: Tell me about your most interesting project.

'I built an intelligent dictionary that evolved through three stages — Trie for autocomplete, FastAPI for the REST backend, then SentenceTransformer for semantic search.'

Q: What engineering challenges did you face?

'Biggest challenge was latency — LLM calls take 800ms. I separated the fast path (Trie + DB, <50ms) from the LLM path, loading explanations asynchronously.'

Q: How does your semantic search work?

'I encoded every vocabulary word as a 384-dim vector using SentenceTransformers and stored them in a FAISS index. User queries are encoded and searched against this index.'

11.3 MVP Checklist for Internship Applications

■	Trie-based autocomplete — working and tested
■	FastAPI with /search, /autocomplete, /word/{word} endpoints
■	SQLite word database (5000+ words minimum)
■	Fuzzy correction working and returning 'Did you mean?'
■	Swagger UI at /docs — instant demo-ability
■	Clear GitHub README with architecture diagram
■	Deployed URL (Railway/Render free tier)
■	FAISS semantic search — strong differentiator
■	RAG pipeline — very impressive, rare in student work

12 FUTURE EXTENSIONS

Voice Input (Whisper API)	OCR Dictionary Lookup
Browser MediaRecorder → audio blob → POST /voice/search → Whisper API transcription → standard search pipeline. ~\$0.0003 per voice query.	User photographs a word in a book → pytesseract (Tesseract OCR) extracts text → dictionary lookup. Useful for language learners in context.
Document Semantic Search	Multilingual Translation Pipeline
Extend vector index to full documents (articles, books, Wikipedia pages). User asks question → semantic search over document chunks → RAG response. Architecture of ChatPDF, NotebookLM.	LibreTranslate (self-hosted, free) or DeepL API: search in Hindi → translate to English → search → translate results back. Cross-language learning mode.
AI Agent for Complex Queries	Personalized Learning Engine
Multi-step agent loop: 'Find 5 words meaning sad and write a poem using all of them' → agent plans synonym_search → collect_results → generate_poem.	Track search history, spaced repetition flashcards, vocabulary progress with streaks. Duolingo-style gamification layered on the existing dictionary.
Distributed Architecture	Advanced Cross-Encoder Reranking
API Gateway → Search/Embedding/Graph/LLM microservices → Weaviate/Qdrant cluster → Redis Cluster → PostgreSQL with read replicas.	Two-stage retrieval: FAISS retrieves top-20 candidates (fast bi-encoder), then cross-encoder scores each pair for final top-5. Pattern used in production Bing, Google, Elastic.

Project Summary

Metric	Value
Estimated Build Time	12–16 weeks (10–15 hrs/week)
Lines of Code	3,000–6,000 (backend) + 1,000–2,000 (frontend)
Core DSA Implemented	Trie, Edit Distance DP, BFS/DFS, Dijkstra, Priority Queue
AI/ML Techniques	Dense Embeddings, Vector Search (FAISS), RAG, Prompt Engineering
APIs & Frameworks	FastAPI, SQLAlchemy, Redis, SentenceTransformers, OpenAI
Deployment Target	Docker + Railway.app/Render.com (free tier)
Resume Impact	Top 5% of undergraduate project portfolios for AI-era SDE roles

Document Version 1.0 | IIT Madras Engineering Project Portfolio | Architecture: DSA Core + REST Backend + AI Semantic Retrieval